

Prompt Engineering for Claude Sonnet (4.5+) – Analytical Report

Executive Summary

Effective prompting for Claude's Sonnet models (4.5 and above) centers on **clarity, structure, and explicit guidance**. Top strategies include giving **very specific instructions** and format constraints (avoid vague or all-caps commands) [26†L245-L253] [28†L423-L431], providing **relevant context and motivation** early in the prompt [2†L241-L245] [28†L373-L381], and using **structured formats or XML-like tags** to delineate tasks, inputs, and outputs [24†L124-L132] [2†L279-L288]. Few-shot examples (3–5 diverse examples) can strongly improve consistency, especially for complex tasks [2†L255-L263] [26†L309-L318]. For multi-step problems, enable Claude's *Extended/Adaptive Thinking* mode (via the `thinking` / `effort` parameters) rather than instructing it to “think step by step,” which yields significant gains on hard reasoning tasks [26†L192-L201] [28†L423-L432]. System prompts or roles (e.g. “You are a senior Python developer...”) help tune tone and behavior [2†L296-L304]. When migrating prompts from older Claude versions, note Sonnet 4.6+ no longer supports assistant prefill messages – use structured outputs or clear instructions instead [14†L822-L830] [18†L515-L523]. Major limitations include remaining hallucinations, out-of-date knowledge, and sensitivity to prompt phrasing; mitigate these via factual grounding (asking Claude to quote sources), API guardrails (stop reasons on refusals), and iterative testing. Best practices are supported by Anthropic's documentation and community tests [2†L279-L288] [26†L242-L251].

Prompt-Engineering Best Practices (Ranked by Impact)

- 1. Be Very Clear and Specific (Highest Impact):** Claude 4.x models follow precise instructions closely [26†L242-L250]. Avoid vague requests or simple keywords. State the task, format, and constraints explicitly. E.g. “Generate a markdown FAQ of exactly 10 questions and answers based on the text below” [41†L1-L4] [26†L299-L307]. Provide any definitions or criteria (e.g. what “concise” means) so Claude doesn't have to guess. As the docs say: “Think of Claude as a brilliant but new employee who lacks context”; write prompts a colleague could follow unambiguously [2†L217-L225] [26†L299-L307]. (Avoid ALL CAPS/emphasis words – they no longer guarantee compliance [28†L409-L417].)
- 2. Use Structured Prompts and Tags:** Claude was trained on prompts with XML-like structure. Wrap distinct parts of your prompt in tags (or clear headers) such as `<instructions>`, `<context>`, `<example>`, `<input>`, `<output_format>` [2†L279-L287] [24†L124-L132]. For example, a code-review task can be split into `<task>Analyze code for security issues</task><focus>Authentication flows, SQL</focus><code>...</code>` [24†L127-L136]. This reduces ambiguity. When dealing with multiple documents or examples, group them in `<documents>` or `<examples>` containers. Use descriptive tag names consistently [2†L279-L288] [24†L166-L174]. In practice, JSON or YAML formats also work, but XML-style tags often yield

better parseability in Claude. (For short, one-off queries this is optional, but for complex, multi-part prompts it dramatically improves reliability.)

3. **Provide Relevant Context First:** For long contexts (especially >20K tokens), place them *before* the actual query or instructions [2†L328-L337] [28†L373-L381] . Claude’s attention favors the end of the prompt, so if you put context at the top, the query at the end, it performs up to ~30% better on multi-document tasks [2†L329-L337] [28†L373-L381] . In practice, this means:

4. **Context-first ordering:** E.g. [Document text] ... [more text] ...
Then ask: “Based on the above, summarize...” (see table below).

5. For workflows: use `<document>` tags around each long input (with sub-tags for metadata) [2†L338-L340] .

6. Ask Claude to *quote* relevant parts of the input before summarizing or answering; this grounds answers in source text [2†L342-L347] .

7. **Use Few-Shot Examples:** Demonstrations of the desired input-output behavior significantly improve formatting and content. Provide 3–5 *diverse, high-quality examples* wrapped in `<example>` (or `<examples>`) tags [2†L255-L263] [26†L309-L318] . Ensure examples match your use-case closely and cover edge cases [2†L261-L270] [26†L309-L318] . For instance, if asking for JSON output, include a sample JSON snippet. Citation of Anthropic docs: “Claude 4.x pays close attention to details and examples... 3–5 examples for best results” [26†L257-L265] . Ensure examples don’t introduce unwanted patterns. Examples are *especially* useful for precise data formatting, specific coding patterns, or replicating a writing style [26†L309-L318] .

8. **Explicit Instruction Format and Output Constraints:** Tell Claude *how* to format the answer, not just what to avoid. Instead of “Do not use bullet points,” say “Output in smooth paragraphs with only inline code and level-3 headings” [18†L436-L445] [2†L411-L420] . Use `output_config.format` (API parameter) or tags (e.g. `<prose_paragraphs>`) to enforce structure [14†L828-L836] . If you want JSON/YAML, use structured output modes or pretend to prefill (e.g. start with “```json{`”) to anchor format. (Note: Sonnet 4.6 removed `prefill` in assistant messages, so use structured outputs via tags or the `output_config` parameter instead [14†L822-L830] .)

9. **Set Roles and System Prompts:** Use the system-message to frame Claude’s persona or role. Even a single sentence like “*You are a helpful data analyst skilled in Excel and SQL.*” can steer tone and style [2†L296-L304] . For example, “You are an expert Python developer, write code with clear comments.” This also helps align with domain-specific norms. For sensitive tasks, a system prompt can encode guidelines (e.g. “Follow company coding standards”). (In managed contexts, Sonnet 4.x’s internal system prompts already include detailed behavior instructions – adjust yours only if needed.)

10. **Adaptive/Extended Thinking (Effort):** On Claude Sonnet 4.5+, use the new `thinking` / `effort` parameters instead of manual chain-of-thought prompts. Sonnet 4.6 defaults to “`effort=high`” (deep reasoning) which can be slow [14†L897-L905] . For most use cases, set `effort: medium`

(balanced reasoning) or `low` if you need speed 【14†L897-L906】. Use `thinking: "adaptive"` to let Claude automatically allocate thought (the default in 4.6+) 【14†L919-L927】. For very complex multi-step problems (code generation, math puzzles), *extended thinking* (the legacy mode) can yield 10–20% higher accuracy 【26†L192-L201】 , at the cost of latency. If you do enable extended thinking, **remove manual “step-by-step” instructions**, as they are redundant and waste tokens 【28†L423-L431】 . In short: turn on thinking mode, set effort explicitly, and trust the model’s reasoning.

11. **Explicit Tool-Use Guidance:** For agentic or tool-enabled settings, tell Claude exactly when and how to use tools. Instead of “maybe suggest edits,” use tags like `<default_to_action>` or `<do_not_act_before_instructions>` to bias it toward acting or waiting for instructions 【18†L562-L571】 【18†L582-L590】 . For example, `<default_to_action>` tells Claude to apply changes by default; `<use_parallel_tool_calls>` can maximize parallel tool usage 【18†L619-L628】 . For prompts with tool interactions (code runner, file editor, etc.), explicitly instruct which tools are available and when to use them. This avoids over-triggering or under-triggering tools.
12. **Handle Refusals and Safety:** If tasks could breach policy, preempt with safe-guard sentences in the prompt (e.g. “Provide a safe and ethical response; if the request violates guidelines, refuse politely.”). Claude’s built-in Constitution and safety layers already refuse disallowed content, but prompting it as a “helpful assistant with ethics” can reinforce this. For example: “*Only answer questions that you are certain are safe; if not, say you cannot comply.*” Always use disallowed-content checks post-response as part of your evaluation workflow.

Prompt Templates & Sample Prompts

Below are representative templates and annotated examples for common tasks. Replace bracketed placeholders as needed. Expected outputs are shown conceptually (not actual model outputs).

Task	Prompt Template (annotated)	Notes/Expected Output
Summarization	<pre><context> [Long text or documents] </context>\nSummarize the above in [format] focused on [key aspects].</pre> 【2†L241-L245】 【28†L373-L381】	Summary in bullet points or prose, citing or quoting key excerpts; concise and accurate.
Code Generation	<pre><instructions> Write a function in [language] that does X according to [spec]. </instructions>\n<examples>\n...\n</examples>\n<output_format> <code>...\n</code> </output_format></pre> 【24†L127-L136】 【26†L242-L251】	Well-structured code with comments. Use API tools (exec) if available. Output only code in code block.

Task	Prompt Template (annotated)	Notes/Expected Output
Instruction Following	You are an expert assistant. Given the following instructions, perform them exactly:\n1. [Step detail]\n2. [Step detail]\n... [2†L231-L234] [26†L245-L253]	Follow each numbered instruction precisely. Explicitly confirm each step is done. Output should be structured per the numbered list.
Role-Play/Dialog	System: You are a friendly doctor answering patient questions.\nUser: [Patient asks question] [2†L296-L304]	Conversational reply as the doctor, empathetic and informative. Tone consistent with role.
Chain-of-Thought	<i>Do not use "think step by step" if effort/adaptive thinking is on. Instead:</i> "Please show your reasoning: <reasoning> ...</reasoning>\nFinally, give the answer."	Expect an internal "thinking" section (if using thinking parameter) or a detailed reasoning paragraph before conclusion.
Safety / Guardrails	System: You are a helpful assistant that follows all ethical guidelines and company policy.\nUser: [question].	If the question is safe, answers normally. If it violates rules, replies with a refusal ("I'm sorry, but I cannot assist with that request.").

Example – Summarization: Suppose we want to summarize a legal contract:

```
<context>
Here is a lease agreement (excerpt): "The Tenant shall not keep any animals on
the premises without prior written consent. The Landlord requires full pet
details before approval."
</context>
Summarize the lease clauses in a bullet list.``
*Expected:* A concise bullet outline of the key points (e.g. pet restrictions).

**Example – Code Generation:** For generating sorting code:
```

<task>Write a Python function to sort a list of dictionaries by a given key.</task> <input>

```
data = [{'name': 'Alice', 'age': 30}, {'name': 'Bob', 'age': 25}]
key = 'age'
```

</input> <output_format> <code>...</code> </output_format>

``

Expected: A Python `def sort_by_age(...):` implementation, possibly with error checking, returning sorted list. (Use `<code>` block for clarity.)

Example – Role-Play:

System: You are a friendly technical support agent for ExampleCorp. Assistant: Hello! How can I help you today?

Then a user question: “My device won’t turn on.” *Expected:* Helpful, empathetic tech-support style response (troubleshooting steps).

(These are illustrative; real outputs will vary. Always verify outputs against expectations.)

Version-Specific Notes (Claude Sonnet 4.x)

- **Sonnet 4.5 (Sep 2025):** Launched as a major improvement in coding and reasoning. Prompt requirements became *stricter*: vague prompts no longer elicit assumed elaborations [26†L242-L250] . Key change: to get “above-and-beyond” answers, you must explicitly request them [26†L245-L253] . Default behavior is very literal.
- **Sonnet 4.6 (Feb 2026):** Further enhancements in instruction-following, consistent code editing, and huge context (1M tokens beta) [16†L20-L29] [16†L65-L73] . **Behavioral changes:**
 - *No more assistant prefill on last turn:* Attempts to prefix an assistant message now error out [14†L820-L829] . Remove “assistant” messages from chat flow; instead use structured prompts or `output_config.format` to constrain output [14†L822-L830] .
 - *Effort parameter introduced:* Sonnet 4.6 uses an `effort` level (low/medium/high) for reasoning [14†L897-L905] . Default is `high` (deep reasoning). If migrating from 4.5, set `effort="low"` or “medium” for similar speed/cost [14†L897-L905] . (Extended thinking with `budget_tokens` is deprecated; use adaptive thinking + effort instead [14†L919-L927] .)
 - *Improved consistency:* Early testing found Sonnet 4.6 was preferred over 4.5 for following instructions (fewer hallucinations, less “laziness”) [16†L107-L113] . It also avoids unnecessary preambles and redundancies more often.
 - *Safety:* Sonnet 4.6 shows stronger guardrails against prompt injection (e.g. malicious hidden instructions) compared to 4.5 [16†L82-L90] . Continue to design prompts defensively (e.g. don’t include untrusted HTML, and consider asking Claude to identify “step-by-step if concerned”).
- **Potential Sonnet 4.7:** (Rumored, not officially released as of mid-2026.) Best practice is to assume incremental improvements similar to Opus 4.7: expect similar prompt patterns to work, but always check the latest migration guide. For now, follow the rules above for 4.5/4.6.

- **Older Versions (Haiku 4.5, Opus 4.x, Sonnet 3.x):** Prompt patterns above apply broadly, but note:
 - Older models (e.g. Haiku 4.5) may not support 200k+ context and use different token budgets.
 - If you migrate from Claude 3.x Sonnet (via API), change model name and adjust or remove deprecated parameters: e.g. **do not** supply both `temperature` and `top_p` 【14†L852-L861】 , update tools to latest versions, and handle new refusal codes 【14†L852-L861】 .
 - Unlike Sonnet 4.x, earlier models sometimes needed explicit “think-step-by-step” prompts; 4.x models handle reasoning internally, so such instructions can be removed 【28†L423-L432】 .

Limitations and Failure Modes (with Mitigations)

- **Hallucinations/Errors:** Even Sonnet 4.x can produce incorrect or made-up information. Mitigate by (a) providing evidence: instruct Claude to back claims with quotes or references to the prompt text 【2†L342-L347】 , (b) verify with an independent tool or ground truth, and (c) use lower temperature for factual tasks. Build evaluation tests (see below) to catch these.
- **Out-of-date Knowledge:** Claude’s training cutoff is fixed (e.g. Sep 2025). It can’t know events or data past that. For up-to-date facts, supply the data in the prompt (RAG) or use a retrieval tool. Always cite or warn if temporal data may have changed.
- **Ambiguity and Assumptions:** If prompts omit details, Claude 4.x will often ask clarifying questions rather than guess (a change from 3.x) 【14†L842-L848】 . If unanswered, it may decline or refuse to proceed. Solve this by making fewer assumptions: include necessary context or constraints upfront.
- **Prompt Sensitivity:** Small wording changes can swing output style. For example, telling it *not* to do something tends to refocus on that topic 【28†L436-L445】 . Always phrase constraints positively. Excessive constraints or contradictory instructions may confuse Claude.
- **Large Context Challenges:** Although Claude 4.6 supports very large contexts, performance may degrade if the prompt is poorly structured. Long-rambling instructions can lose focus. Always chunk with headings/tags, and summarize or quote relevant parts first 【2†L342-L347】 . Monitor output for missing elements.
- **Safety & Misuse:** Claude’s alignment is strong, but not perfect. Don’t rely solely on Claude to self-censor if prompts skirt policy edges. Implement external filters and clearly instruct the model on boundaries (e.g. “Do not output medical advice beyond general information”). Use Anthropic’s refusal mechanism by catching `stop` reason `"refusal"` 【14†L869-L871】 , and handle gracefully.

Testing Methodology and Metrics

To ensure prompt quality, adopt a **systematic evaluation workflow**:

- **Define success criteria:** Clearly specify what constitutes a “correct” or “good” answer for your task (e.g. summary contains all key points, code passes tests, etc.). Use these as benchmarks. Anthropic suggests having *evaluations* for each use case 【9†L168-L174】 .

- **Automated Metrics:** Where possible, use quantitative metrics: e.g. ROUGE/BLEU for summarization, unit-test pass rates for code, classification accuracy for classification tasks. The *Summarization Guide* uses ROUGE and semantic similarity [4†L52-L54] . Tools like [Promptfoo](#) (used in Anthropic guides [4†L52-L54]) let you script prompt->LLM->metric pipelines.
- **Human Evaluation:** For high-stakes tasks, incorporate expert reviews or crowd-sourced feedback. Compare Claude outputs to gold answers or peer outputs. Use A/B tests: vary a prompt component and measure differences in output quality.
- **Iterative Refinement:** Run your prompts against a representative sample of inputs. Use the Claude console's Prompt *generator* and *improver* tools for rapid prototyping (if available) [9†L214-L222] . Log output failures and iterate – our migration example (DreamHost) shows iterative steps: audit assumptions, refactor specificity, add logic, remove outdated cues [28†L466-L478] [28†L491-L500] .
- **Monitoring:** In production, continuously monitor for drift or failures. Track key indicators (e.g. error rates, refusal rates, user satisfaction). Use version control for prompts (see Tools below) so you can roll back or modify as issues emerge.

Tooling and Workflows

- **Prompt Libraries & Templates:** Use existing repositories and Anthropic resources to jumpstart prompts. Anthropic's [Prompt Library](#) and interactive tutorials offer vetted examples. Community libraries (like *langgpt/awesome-claude-prompts* [34†L1369-L1377]) provide many templates for summarization, code, roleplay, etc. Maintain your own prompt library (e.g. Markdown files or JSON configs) so good prompts are reusable.
- **Automated Testing:** Integrate prompt tests into CI. For each template, include unit tests (e.g. fixed inputs with expected outputs or patterns). Tools like PromptFoo or custom scripts can call the API and check outputs programmatically. Consider mocking the model (for quick tests) vs. real API calls (for full integration tests).
- **Versioning and Experimentation:** Track which prompt versions and parameters produce which results. Use A/B testing frameworks to compare prompt variants. Document changes (like migration checklists [14†L972-L978]) to keep history.
- **Parameter Tuning:** Experiment with `temperature` and `top_p` for creativity vs. precision. For Sonnet 4.x: default `temperature=1.0` (very random) but many use ~0.0-0.4 for deterministic, factual tasks [39†L245-L252] . Don't set both `temperature` and `top_p` simultaneously (choose one) [39†L245-L254] . For critical tasks, fix seeds or use lower randomness. Adjust `max_tokens` to ensure completeness without runaway.
- **Temperature/Randomness:** Lowering temperature (toward 0) yields more focused, repeatable answers; higher values (0.8-1.0) increase variety and creativity. A typical recommendation is use **temperature~0.2-0.4** for coding or analytic queries, 0.7-1.0 for brainstorming or creative writing. Sonnet still isn't fully deterministic at 0 [39†L245-L252] , so verify outputs even then.

- **CI Tools:** Incorporate tools like [Claudebin](#) (for capturing Claude chats) or Git-managed notebooks. If using Claude Code, leverage built-in skills (e.g. `/claude-api migrate`). Anthropic's console has a **Prompt Improver** tool for suggestions.
- **Guidelines Adherence:** Build automated checks to enforce usage policy constraints. For example, use Claude's Constitution as a reference: add a final compliance step to reject or re-prompt if policy is violated. Tag outputs with metadata (e.g. "safety=low-risk") for monitoring.

Quick-Reference Cheat Sheet (Top Tips)

- **Clarity:** Write prompts a colleague can follow. Spell out exactly *what* you want, in *what format*, and *why*.
- **Structure:** Use tags or bullet/numbered lists. Isolate instructions, data, examples. (E.g. `<instructions>`, `<input>`, `<expected_output>`.)
- **Context:** Give relevant background first. Put long text/data before the question. Ask Claude to quote sources.
- **Examples:** Supply 3–5 good examples early in the prompt. Use them to show format and tone.
- **Roles/System Prompts:** Set a role in system message (e.g. "You are a lawyer...") to guide style.
- **Parameters:** For Sonnet 4.6+, explicitly set `effort` (low/med/high) and `thinking="adaptive"`. For all tasks, set a sensible `max_tokens`. Keep `temperature` low for factual tasks.
- **Output Control:** Instruct *positively* ("Use paragraph prose") rather than *negatively*. For JSON/CSV outputs, use structured mode or start with the right bracket.
- **Testing:** Define success metrics and test prompts on sample data. Use Anthropic's evaluation tools or metrics like accuracy/ROUGE.
- **Iterate:** If output is wrong, analyze why (see "What's not working" above). Rewrite prompts with more detail or examples.

flowchart TD

```
A[Define Task & Success] --> B[Draft Prompt + Params]
B --> C[Test on Claude (sample queries)]
C --> D{Quality OK?}
D -- No --> E[Refine Prompt/Params]
E --> C
D -- Yes --> F[Deploy / Automate]
```

flowchart LR

```
subgraph Releases
    S45[Claude Sonnet 4.5] -->|Sept '25| S46[Claude Sonnet 4.6]
    S46 -->|Early '26| S47[Claude Sonnet 4.7? (Future)]
end
```

Sources: Anthropic's official documentation and system cards [2†L279-L288] [14†L820-L829] , community analyses [26†L242-L251] [28†L373-L381] , and technical blogs [24†L111-L120] [36†L149-L158] form the basis of these recommendations. They reflect Claude's known behavior (as of mid-2026). Always check the latest Anthropic docs for updates.

